

Staff / Senior Machine Learning Engineer

System Design Interview Preparation Guide

Target Role: Staff / Senior ML Engineer — Xero

10 System Design Questions — 5 Classical ML, 5 AI Agent / LLM / RAG

Prepared by: Principal/Staff ML Interview Coach

How to Use This Guide

This guide is built around the actual interview format you should expect: a whiteboard system design session focused on the production boundary — what it takes to get an ML or AI agent system from a research prototype into a safe, observable, cost-aware production system. The interviewer is explicitly not testing model internals or algorithm trivia; they are testing whether you can design the infrastructure, data flow, and team/ownership boundaries around a model.

Each question below includes: the context of what it is actually probing, a full solution architecture diagram you should be able to reproduce on a whiteboard from memory (not verbatim, but the components and the reasoning), a complete spoken-style answer of 250+ words, and a "Key Takeaways" box capturing the specific signals an interviewer is listening for. Practice talking through each diagram out loud, component by component, narrating the data flow and explicitly calling out where research ends and production begins.

The ten questions are split into two sets of five: classical ML system design (fraud detection, categorization, feature stores, recommendations, monitoring/retraining) and AI agent / LLM / RAG system design (RAG assistants, autonomous agents, document extraction, conversational support, multi-agent platforms). Both sets are calibrated to Xero's actual product surface — accounting, payments, bank reconciliation, and small business financial workflows — because the strongest answers in these interviews are ones that show you understand the specific domain, not just generic ML system design patterns.

What "Staff" Signals Look Like in These Answers

- Platform thinking: designing shared infrastructure (feature stores, monitoring, registries) that serves many models/teams, not just one model
- Risk-tiered decisions: explicitly varying automation, approval gates, and rollout caution based on the financial/safety stakes of each system
- Naming the research/production boundary explicitly: what promotion gate, review, or eval bar separates a notebook experiment from a deployed model
- Talking about failure modes and degraded operation, not just the happy path
- Tying technical design back to who owns what — on-call, model cards, cross-team contracts — because Staff engineers are evaluated on organizational leverage, not just technical depth

Table of Contents

[How to Use This Guide](#)

[Part 1: Classical ML System Design](#)

[ML Question 1: Design a real-time fraud / anomaly detection system for payments and invoices](#)

[ML Question 2: Design a multi-tenant bank transaction auto-categorization system at scale](#)

[ML Question 3: Design a feature store and training pipeline for a churn or credit-risk scoring model](#)

[ML Question 4: Design a real-time recommendation system for matching bank transactions to invoices](#)

[ML Question 5: Design a model monitoring, drift detection, and automated retraining platform across many production models](#)

[Part 2: AI Agent / LLM / RAG System Design](#)

[Agent Question 1: Design an "ask your accounts" RAG assistant over a customer's financial data](#)

[Agent Question 2: Design an autonomous bookkeeping agent that performs multi-step actions with tool use](#)

[Agent Question 3: Design a scaled, production LLM-powered document extraction pipeline for invoices and receipts](#)

[Agent Question 4: Design a conversational customer support agent combining RAG, account-data tools, and human escalation](#)

[Agent Question 5: Design a multi-agent orchestration platform with a router, specialist agents, shared memory, and safety controls](#)

[Whiteboard Session: Delivery Tips](#)

Part 1: Classical ML System Design

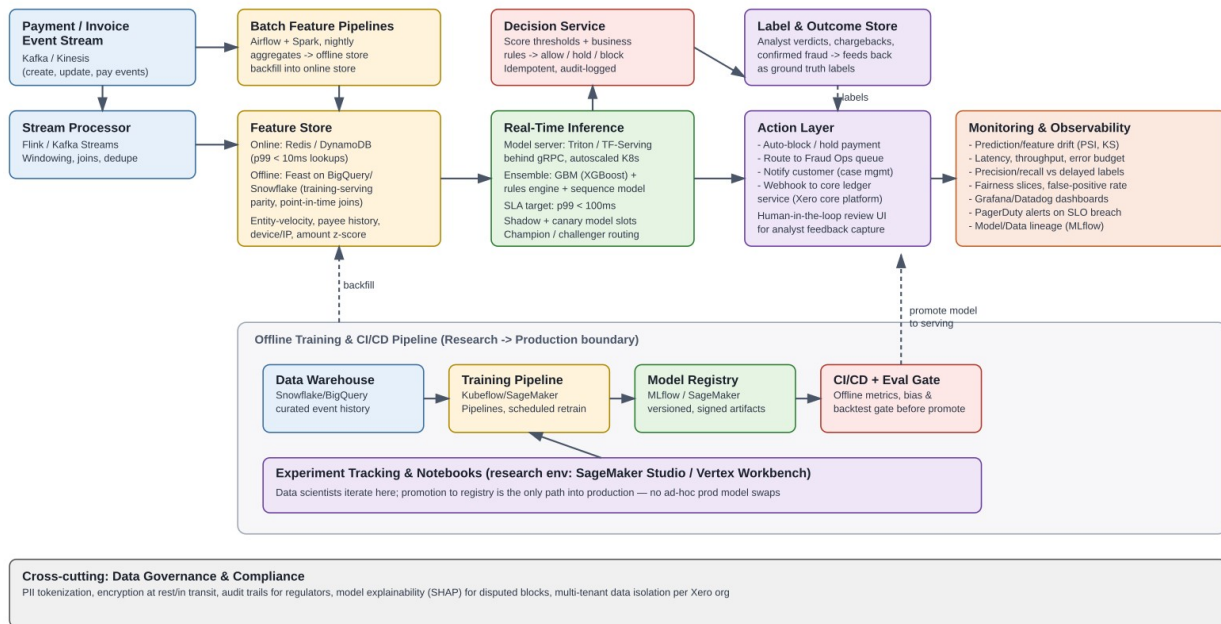
These five questions cover the core ML system design surface area: real-time inference, feature stores, batch + online hybrid systems, recommendation architectures, and the MLOps platform layer that ties them together. Each is set in a Xero-relevant domain context (payments, bank feeds, churn/credit risk) because interviewers at this level expect you to reason about the specific business, not abstract textbook ML.

ML Question 1: Design a real-time fraud / anomaly detection system for payments and invoices

Tests: streaming architecture, low-latency serving, feature engineering for fraud, the research-to-production boundary, and how you handle delayed/noisy labels — a recurring theme at fintech-adjacent companies like Xero.

Solution Architecture

Real-Time Payment Fraud / Anomaly Detection — End-to-End Architecture



Full Answer

I'd design this as a streaming-first system with a clear separation between the low-latency online path and the offline training/governance path. Payment and invoice events land on a Kafka/Kinesis topic and are processed by a stream processor (Flink) that performs windowing, deduplication, and joins. Features split into two stores: an online store (Redis/DynamoDB, sub-10ms lookups) for real-time entity velocity, payee history, and device/IP signals, and an offline store (Feast on Snowflake/BigQuery) for training, with identical transformation logic to avoid train-serve skew.

Inference runs as an ensemble — a gradient-boosted model plus a rules engine plus a sequence model — served behind a model server (Triton/TF-Serving) on autoscaled Kubernetes with a p99 target under 100ms. A decision service applies score thresholds and business rules to output allow/hold/block, then an action layer either auto-blocks, routes to a fraud-ops review queue, or notifies the customer. Analyst verdicts and confirmed chargebacks feed back as ground-truth labels, which is the hard part of fraud specifically — labels arrive late

and are sparse, so I'd track a rolling backtest against delayed labels rather than pretending real-time accuracy is knowable in real time.

Training happens offline via Kubeflow/SageMaker Pipelines, gated by an evaluation step (bias check, backtest against historical fraud, latency budget on sample inference) before promotion through a model registry into a shadow/canary slot, then full traffic. I'd insist on SHAP-based explainability stored per decision, since blocked payments need to be explainable to both customers and regulators. Monitoring covers feature/prediction drift (PSI/KS), latency, and fairness slices across customer segments — this is where most fraud systems quietly degrade without anyone noticing for months.

Key Takeaways — What the Interviewer Is Listening For

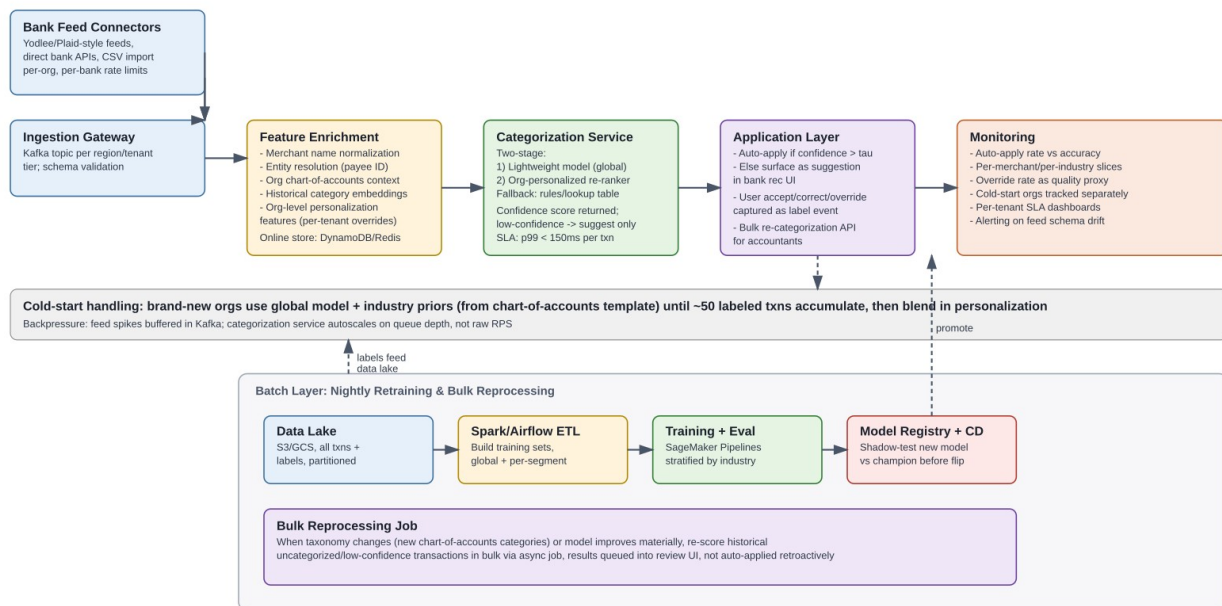
- Separate low-latency online serving from offline training; keep feature logic identical across both to avoid train/serve skew
- Fraud labels are delayed and sparse — design monitoring around delayed-label backtests, not real-time accuracy
- Ensemble of rules + ML gives you a fast, auditable fallback when the model is uncertain or down
- Explainability (SHAP) is not optional here — disputed blocks need a reason a human can act on
- Promotion through a registry with an automated eval gate is the boundary between research and production

ML Question 2: Design a multi-tenant bank transaction auto-categorization system at scale

Tests: multi-tenant ML architecture, personalization vs global models, cold-start handling, and designing for graceful degradation — core to Xero's actual product surface.

Solution Architecture

Bank Transaction Auto-Categorization — Multi-Tenant, Batch + Online



Full Answer

The core tension here is global accuracy versus per-organization personalization, at the scale of millions of small businesses each with their own chart of accounts and spending patterns. I'd use a two-stage model: a lightweight global model trained across all tenants for broad pattern recognition, plus an org-personalized re-ranker that adjusts based on that organization's own historical corrections. A rules/lookup fallback handles well-known merchants deterministically and cheaply, so the ML model is only invoked where it adds value.

Architecturally, bank feed connectors ingest transactions per-org into Kafka, which feeds a feature enrichment stage doing merchant normalization, entity resolution, and pulling org-specific embeddings from an online feature store. The categorization service returns a confidence score; above a threshold the category auto-applies, below it the system surfaces a suggestion in the bank-reconciliation UI instead of guessing. Every accept, reject, or manual override is captured as a labeled feedback event — this is the dataset that actually drives the personalization layer over time.

Cold start is a first-class design concern, not an edge case: brand-new organizations get the global model plus industry priors derived from their chart-of-accounts template, blending toward personalization only after enough labeled transactions accumulate. I'd track auto-apply rate

against accuracy and override rate as a quality proxy, sliced separately for cold-start orgs so a strong aggregate metric doesn't mask a bad new-user experience. Nightly batch retraining handles both the global model and segment-specific variants, with bulk reprocessing jobs available for when the category taxonomy itself changes — those results go to a review queue rather than auto-applying retroactively.

Key Takeaways — What the Interviewer Is Listening For

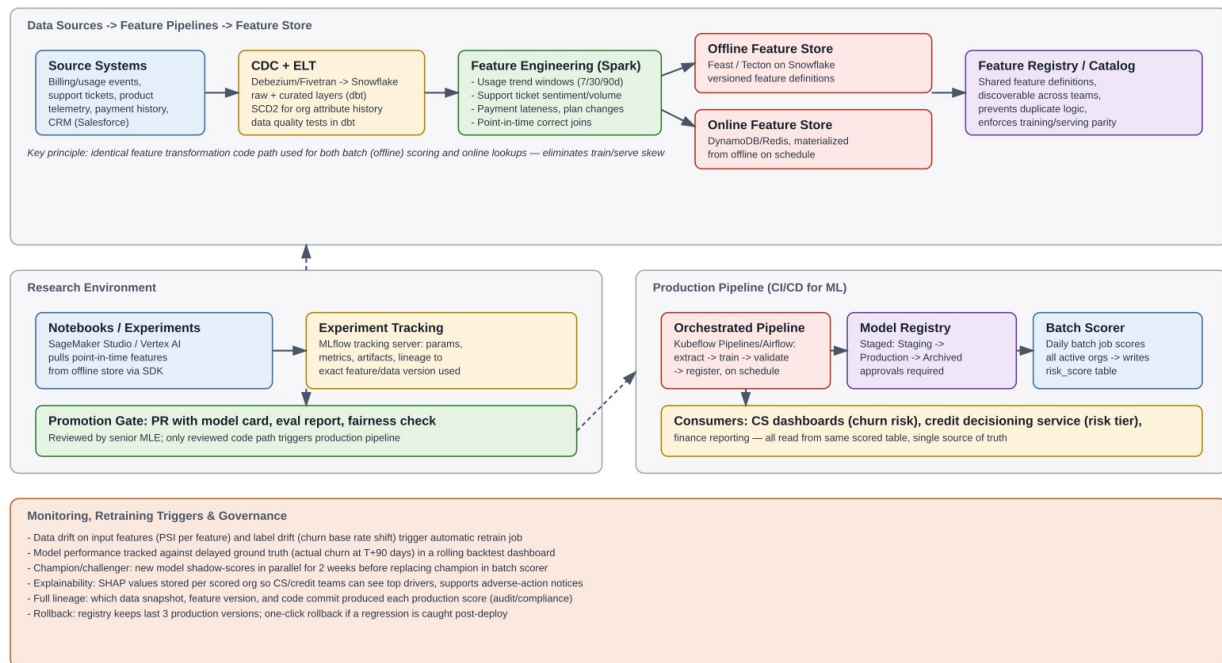
- Two-stage design (global model + personalized re-ranker + rules fallback) balances cold-start coverage with per-tenant accuracy
- Confidence-gated auto-apply vs suggest-only UX is the safety valve — never force a guess on a financial record
- User corrections are the most valuable label source; design the feedback capture path deliberately
- Cold start needs its own metrics slice; aggregate accuracy will hide it
- Taxonomy changes require bulk reprocessing into a review queue, never silent retroactive reclassification

ML Question 3: Design a feature store and training pipeline for a churn or credit-risk scoring model

Tests: feature store design, training/serving parity, point-in-time correctness, model governance, and the contract between a research environment and a production pipeline.

Solution Architecture

Churn / Credit-Risk Scoring — Feature Store & Training-to-Production Pipeline



Full Answer

I'd start from the principle that the feature store, not the model, is the piece of infrastructure that has to be right — most production ML failures I've seen trace back to train/serve skew or point-in-time leakage in features, not model choice. Source systems (billing, support tickets, product telemetry, payment history) flow through CDC and ELT into a curated warehouse layer, with SCD2 history so I can reconstruct exactly what an organization's attributes looked like at any past date. Spark-based feature engineering computes windowed aggregates and writes to both an offline store (Feast on Snowflake, used for training with point-in-time-correct joins) and an online store (DynamoDB/Redis, materialized from the same definitions for serving).

The research environment — notebooks, experiment tracking via MLflow — pulls features through the same SDK that production uses, so a data scientist's training run and the production scorer are guaranteed to see identical transformation logic. Promotion into production isn't a model file copied somewhere; it's a pull request with a model card, evaluation report, and fairness check, reviewed by a senior engineer, which is the only path that triggers the production pipeline. That pipeline retrains on a schedule or drift trigger, validates against the current champion, and registers a new version with full lineage — which data snapshot, which feature version, which code commit.

For consumption, I'd insist on a single scored table that customer success dashboards, credit decisioning, and finance reporting all read from, rather than each team calling the model independently — one source of truth avoids three different "churn risk" numbers existing in the business simultaneously. Explainability via stored SHAP values supports both CS conversations and any regulatory adverse-action requirements on the credit side.

Key Takeaways — What the Interviewer Is Listening For

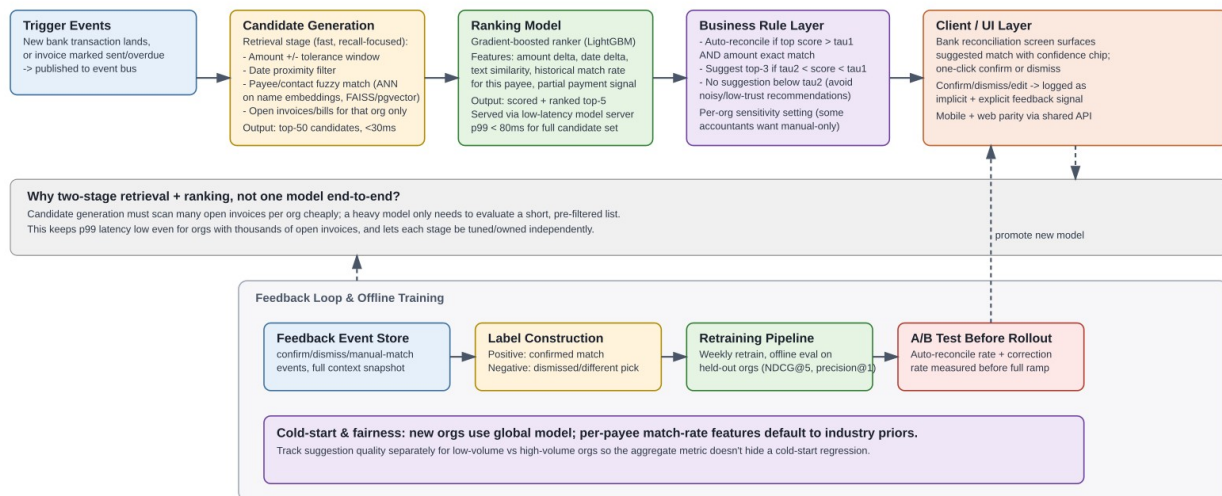
- The feature store's job is enforcing training/serving parity and point-in-time correctness — get this wrong and no model architecture saves you
- Promotion to production should be a reviewed, gated process (PR + model card + eval report), never an ad-hoc model swap
- One scored table as single source of truth prevents multiple teams from quietly diverging on 'the' churn score
- Full lineage (data + feature + code version) per production score is a compliance and debugging necessity, not nice-to-have
- Champion/challenger shadow testing before replacing a production scorer catches regressions before they cost money

ML Question 4: Design a real-time recommendation system for matching bank transactions to invoices

Tests: low-latency retrieval+ranking architecture, candidate generation strategies, designing for variable per-tenant scale, and feedback-loop design for implicit/explicit signals.

Solution Architecture

Smart Bank-Transaction-to-Invoice Matching — Recommendation Architecture



Full Answer

This is a classic two-stage retrieval-and-ranking problem, and the two-stage split matters specifically because organizations vary wildly in scale — some have five open invoices, others have thousands. A single end-to-end model scoring every possible pair doesn't scale cleanly across that range. So candidate generation runs first and cheaply: filter by amount tolerance, date proximity, and a fuzzy payee-name match using approximate nearest-neighbor search over name embeddings, scoped to that organization's open invoices only. That produces maybe fifty candidates in under 30ms.

A ranking model, a gradient-boosted ranker, then scores those candidates using richer features — amount delta, date delta, text similarity, and crucially the historical match rate for that specific payee, which captures patterns like "this customer always pays three days late." The output feeds a business rule layer: auto-reconcile only if the top score clears a high threshold and the amount matches exactly; suggest the top three if it's in a middle confidence band; show nothing if confidence is too low, because a wrong suggestion erodes trust faster than no suggestion does. I'd also expose a per-organization sensitivity setting, since some accountants explicitly want manual control over reconciliation regardless of model confidence.

The feedback loop is the most operationally important part: every confirm, dismiss, or manual override becomes a labeled training example, retrained weekly and evaluated offline with ranking metrics like NDCG@5 before any A/B rollout. I'd specifically track suggestion quality separately for low-volume versus high-volume organizations, because aggregate metrics dominated by power users can hide a cold-start problem for everyone else.

Key Takeaways — What the Interviewer Is Listening For

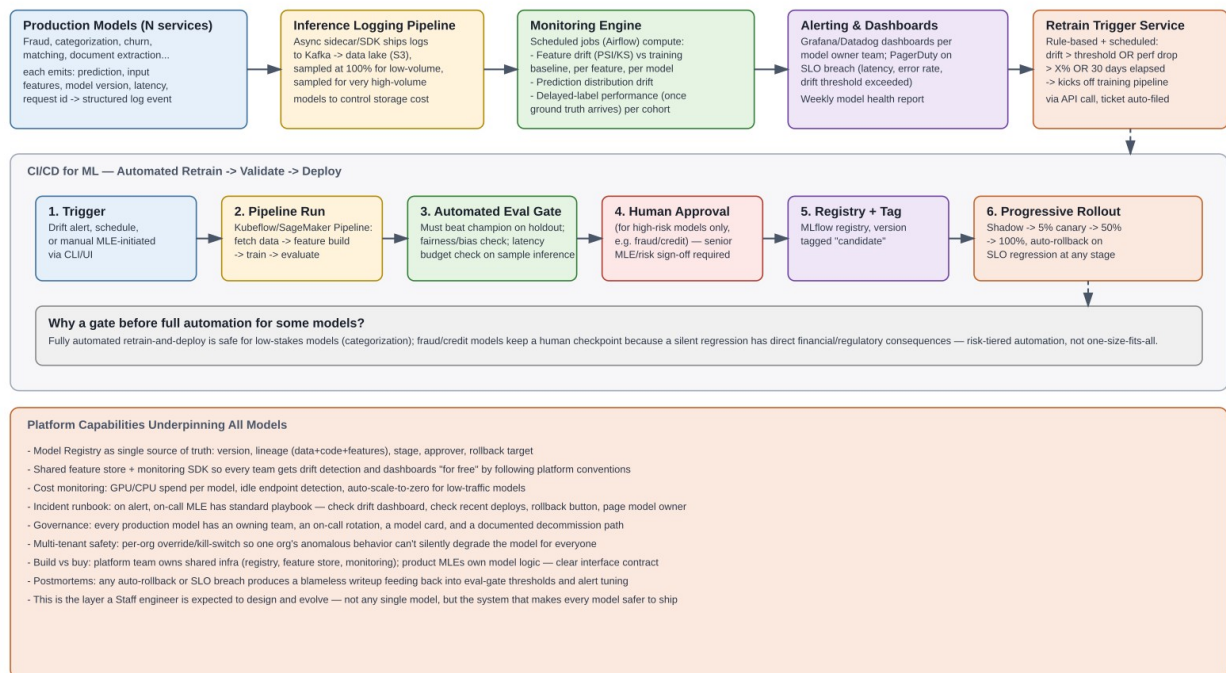
- Two-stage retrieval+ranking exists specifically to handle wildly variable per-tenant scale without sacrificing latency
- Confidence-banded actions (auto-apply / suggest / nothing) protect user trust better than always showing a best guess
- Per-payee historical match rate is a high-signal feature that a generic similarity score alone would miss
- Confirm/dismiss feedback is implicit labeled data — design the event capture deliberately from day one
- Segment evaluation metrics by usage volume; aggregate metrics can hide cold-start or low-volume regressions

ML Question 5: Design a model monitoring, drift detection, and automated retraining platform across many production models

Tests: platform-level thinking (the hallmark of Staff level), risk-tiered automation, MLOps/CI-CD design, and how you'd build infrastructure that scales across teams rather than one model.

Solution Architecture

Multi-Model Monitoring, Drift Detection & Automated Retraining Platform



Full Answer

This question is really asking whether I can design infrastructure that serves many models and many teams, not just solve one model's monitoring problem — that's the Staff-level distinction. Every production model emits a structured inference log (prediction, input features, model version, latency) through an async logging pipeline into a shared data lake, sampled appropriately by volume to control storage cost. A monitoring engine runs scheduled jobs computing feature drift (PSI/KS against the training baseline), prediction distribution drift, and — once ground truth arrives — actual performance per cohort. Results surface on per-team dashboards with PagerDuty alerts on SLO breach.

The CI/CD pipeline for ML is the part I'd spend the most design time on: a retrain trigger (drift threshold, performance drop, or schedule) kicks off an orchestrated pipeline that fetches data, builds features, trains, and runs an automated evaluation gate — the new model must beat the champion on a holdout set and pass fairness and latency checks. For high-stakes models like fraud or credit risk, I'd add a mandatory human approval step before promotion; for lower-stakes models like categorization, I'd let it deploy automatically. That's a deliberate risk-tiering decision, not a blanket policy, because a silent regression in a low-stakes model costs little while one in a credit decision has real financial and regulatory consequences.

Rollout itself is progressive — shadow, then a small canary, then a larger percentage, then full traffic — with automatic rollback on SLO regression at any stage. The platform-level capabilities underneath all of this — a shared model registry as the single source of truth, a shared feature store, cost monitoring per model, and a standard incident runbook — are what let dozens of models across many teams get monitoring and safe deployment "for free" by following platform conventions, rather than each team reinventing this.

Key Takeaways — What the Interviewer Is Listening For

- The deliverable at Staff level is the platform that makes every model safer to ship, not any single model's pipeline
- Risk-tiered automation (full auto-deploy vs human-gated) should be a deliberate policy decision per model class
- Progressive rollout (shadow -> canary -> full) with automatic rollback is the safety net that makes frequent retraining viable
- Shared registry + feature store + monitoring SDK reduce the marginal cost of onboarding the Nth model to near zero
- Postmortems on any auto-rollback should feed back into eval-gate thresholds — the platform should get smarter from its own incidents

Part 2: AI Agent / LLM / RAG System Design

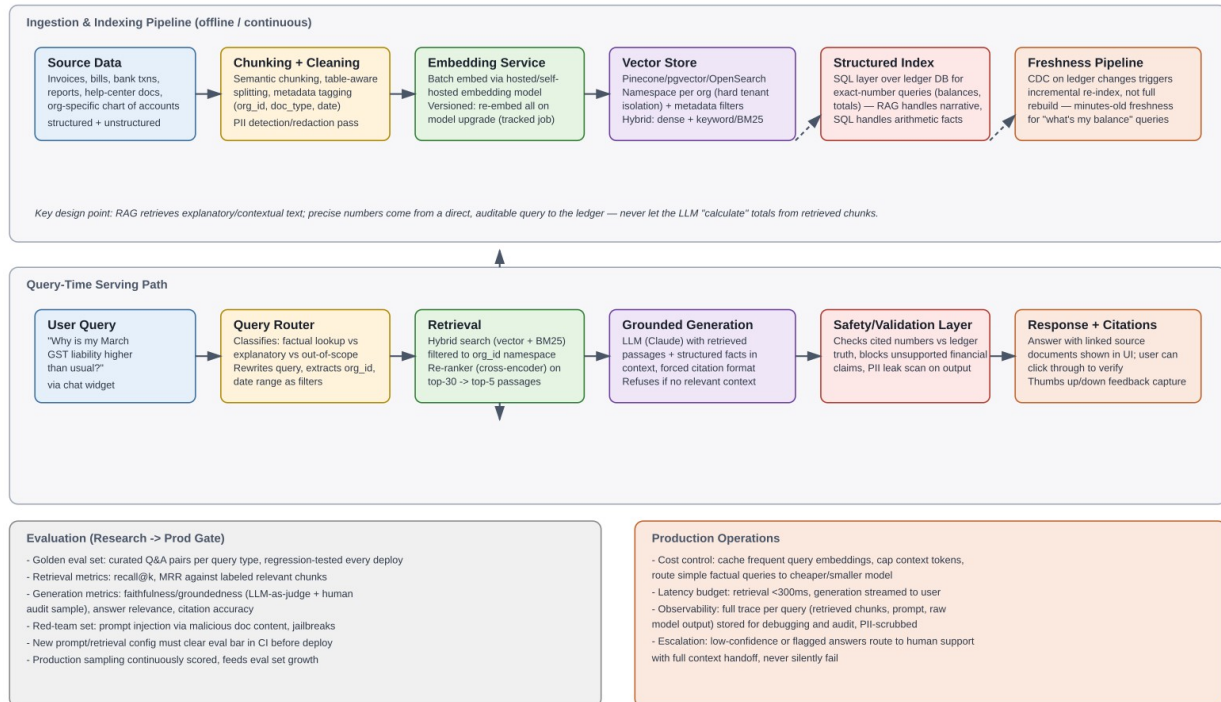
These five questions cover agentic and LLM-powered system design: retrieval-augmented generation, autonomous multi-step agents with tool use, LLM-based extraction pipelines, conversational support agents, and multi-agent orchestration platforms. The throughline across all five is safety and groundedness — these systems can take real actions or state real numbers with confidence, so the architecture has to be built around guardrails, escalation, and evaluation from the start, not bolted on afterward.

Agent Question 1: Design an "ask your accounts" RAG assistant over a customer's financial data

Tests: RAG architecture end-to-end, multi-tenant vector store isolation, the critical RAG-vs-SQL distinction for numeric accuracy, and groundedness evaluation — a frequent AI-agent question at companies handling financial data.

Solution Architecture

"Ask Your Accounts" — RAG Assistant Over Financial Data, Scratch to Production



Full Answer

The single most important design decision here is recognizing that RAG should answer the "why" and a direct database query should answer the "what." If a customer asks why their GST liability is higher this month, retrieval surfaces relevant context — transaction narratives, prior categorizations, help articles on GST rules — but the actual liability number comes from a direct, auditable SQL query against the ledger, never from the LLM "calculating" a total out of retrieved text chunks. Conflating those two is how RAG systems hallucinate numbers that look plausible and are wrong.

For ingestion, source documents (invoices, reports, help-center content) go through chunking that's table-aware and metadata-tagged with org_id and date, then embedded and stored in a vector store namespaced per organization for hard tenant isolation — never a shared index with a metadata filter as the only boundary, since a filter bug becomes a cross-tenant data leak. A CDC-driven freshness pipeline does incremental re-indexing on ledger changes so balance-related answers stay current within minutes rather than waiting for a nightly rebuild.

At query time, a router classifies the question type and extracts org_id and date filters, hybrid (vector plus keyword) retrieval pulls candidates from that org's namespace only, a cross-encoder re-ranks down to the top few passages, and generation happens with retrieved passages and structured facts both in context, with a forced citation format and an explicit instruction to refuse rather than guess when context is insufficient. A validation layer checks any cited numbers against ledger truth before the response ships. Evaluation runs a golden Q&A set for groundedness and citation accuracy plus a red-team set for prompt injection via malicious document content, and production responses get sampled and scored continuously to grow that eval set over time.

Key Takeaways — What the Interviewer Is Listening For

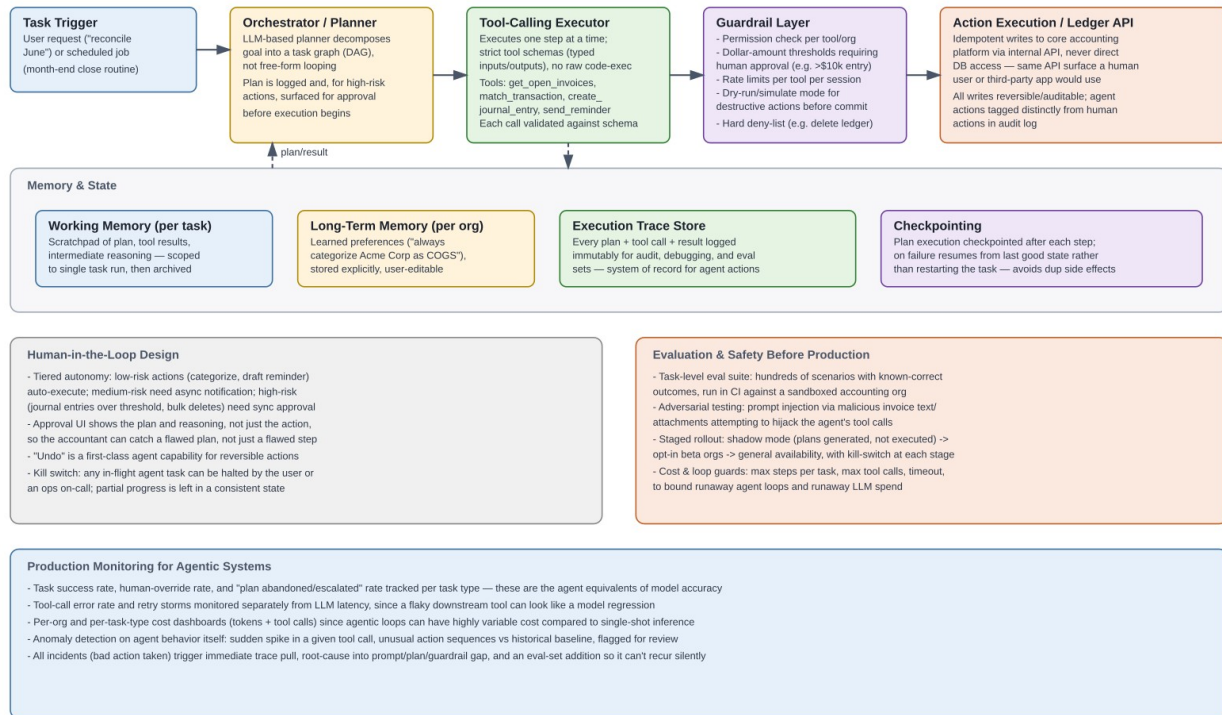
- RAG handles narrative/explanatory content; precise financial numbers should come from a direct, auditable query — never let the LLM compute totals from retrieved text
- Per-org vector namespace isolation is a hard tenant boundary; a metadata filter alone is not sufficient isolation for financial data
- CDC-driven incremental re-indexing keeps balance-sensitive answers fresh without full reindex cost
- Forced citations plus a validation layer that checks numbers against ground truth is the core anti-hallucination control
- Golden eval set + red-team prompt-injection set are both required gates before any prompt or retrieval change ships

Agent Question 2: Design an autonomous bookkeeping agent that performs multi-step actions with tool use

Tests: *agentic planning architecture, guardrail design, tiered human-in-the-loop approval, and safety controls for an agent that can take real, financially consequential actions — this is the question most likely to probe production-readiness depth.*

Solution Architecture

Autonomous Bookkeeping Agent — Multi-Tool Agentic Workflow with Guardrails



Full Answer

The architecture I'd push for is a planner that produces an explicit task graph up front, rather than a free-form reasoning loop that decides each step as it goes — because an inspectable plan is something a human or an automated guardrail can review before any action happens, while a pure ReAct-style loop is much harder to reason about or bound. The orchestrator decomposes a goal like "reconcile June" into discrete steps, logs the plan, and for high-risk actions surfaces it for approval before execution begins at all.

Execution happens one tool call at a time against strict, typed schemas — get_open_invoices, match_transaction, create_journal_entry — never raw code execution, and every call passes through a guardrail layer enforcing per-tool permissions, dollar-amount thresholds that require human approval, rate limits, and a hard deny-list for genuinely irreversible actions. All writes go through the same internal ledger API a human user would use, never direct database access, and agent-originated actions are tagged distinctly in the audit log from human-originated ones.

I'd design autonomy as tiered rather than binary: low-risk actions like categorization auto-execute, medium-risk actions get async notification, and high-risk actions like large journal entries need synchronous approval — critically, the approval UI shows the reasoning and plan, not just the proposed action, so an accountant can catch a flawed plan rather than only a flawed final step. Before production, I'd run a task-level eval suite of hundreds of scenarios against a sandboxed org, adversarial testing for prompt injection via malicious invoice content attempting to hijack tool calls, and a staged rollout — shadow mode generating but not executing plans, then opt-in beta, then general availability — with a kill switch and bounded max-steps-per-task at every stage to prevent runaway loops.

Key Takeaways — What the Interviewer Is Listening For

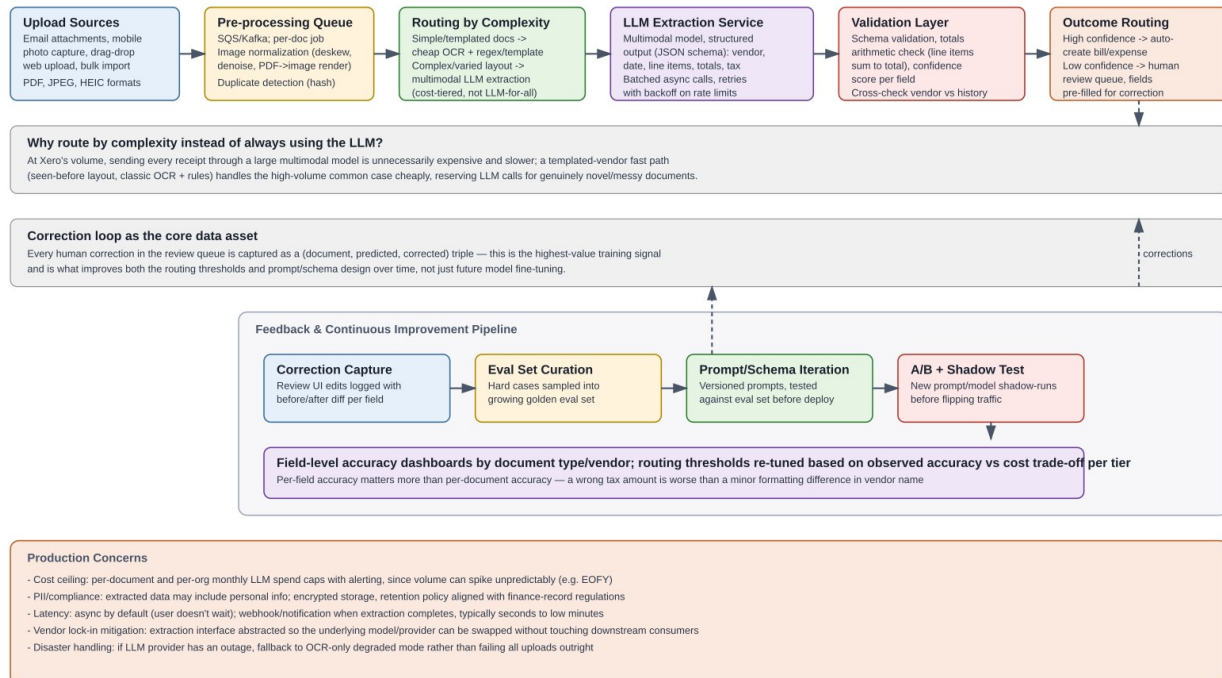
- An explicit, logged task-graph plan is far more reviewable and safer than a free-form agentic loop deciding steps live
- Guardrails (permission checks, dollar thresholds, rate limits, deny-lists) belong in a dedicated layer the agent can't bypass, not embedded in prompts
- Tiered autonomy (auto / async-notify / sync-approve) matched to action risk is the practical alternative to all-or-nothing automation
- Approval UIs should show the plan and reasoning, not just the final action, so humans can catch upstream planning errors
- Staged rollout (shadow -> beta -> GA) with a kill switch and step/cost caps is non-negotiable before agents can take real financial actions

Agent Question 3: Design a scaled, production LLM-powered document extraction pipeline for invoices and receipts

Tests: cost-aware routing architecture, structured output validation, the correction-loop-as-data-asset concept, and graceful degradation when an LLM provider has issues — common at any company processing high document volume.

Solution Architecture

LLM-Powered Invoice/Receipt Extraction — Scaled Production Pipeline



Full Answer

At meaningful volume, the design question isn't "how do we use an LLM to extract invoice data" — it's "how do we avoid sending every document through an expensive multimodal model when most of them don't need it." I'd route by complexity: documents from previously-seen vendor templates go through a cheap OCR-plus-rules fast path, while genuinely novel or messy layouts go to an LLM extraction service that returns structured JSON against a defined schema — vendor, date, line items, totals, tax. That tiering is what keeps cost and latency sane at scale rather than uniformly expensive.

After extraction, a validation layer does schema validation, an arithmetic sanity check that line items sum to the stated total, and a confidence score per field, cross-checked against that vendor's history. High-confidence results auto-create the bill or expense; low-confidence results route to a human review queue with fields pre-filled for correction rather than blank, which is both better UX and the source of the single most valuable data asset in this system: every correction is captured as a (document, predicted, corrected) triple, feeding both eval-set growth and routing-threshold tuning over time.

Operationally, I'd track field-level accuracy rather than document-level, because a wrong tax amount is a materially worse error than a minor vendor-name formatting difference, and aggregate document accuracy hides that distinction. I'd also abstract the extraction interface from the specific model provider so it can be swapped without touching downstream consumers, enforce per-org and per-document cost caps with alerting since volume spikes unpredictably around things like EOFY, and build a degraded fallback to OCR-only extraction if the LLM provider has an outage, so uploads keep flowing instead of failing outright during exactly the kind of incident that's hardest to recover from gracefully.

Key Takeaways — What the Interviewer Is Listening For

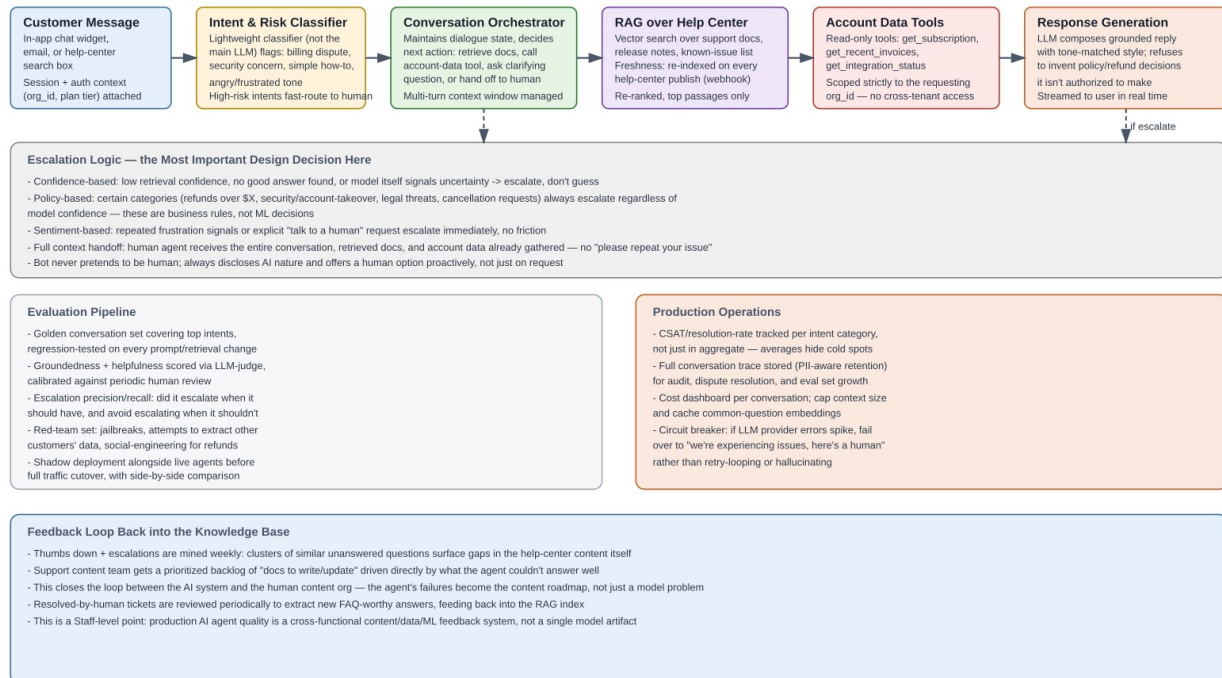
- Route by document complexity — cheap OCR/template path for known vendors, LLM only for novel layouts — to control cost at scale
- Validate structured output with both schema and arithmetic checks (line items sum to total) before trusting it
- Human corrections in the review queue are the highest-value training signal; design that capture path deliberately as (predicted, corrected) pairs
- Track field-level accuracy, not document-level — not all fields carry equal financial risk if wrong
- Plan an explicit degraded mode (OCR-only fallback) for LLM provider outages rather than letting uploads fail outright

Agent Question 4: Design a conversational customer support agent combining RAG, account-data tools, and human escalation

Tests: escalation policy design (the make-or-break decision for support agents), tool-scoped data access for multi-tenant safety, and the cross-functional feedback loop between an AI system and a human content/support org.

Solution Architecture

Customer Support Conversational Agent — RAG + Tools + Human Escalation



Full Answer

The single decision that determines whether this system is trustworthy is the escalation policy, so I'd design that first and treat everything else as supporting infrastructure. Escalation triggers on three independent axes: confidence-based — low retrieval confidence or explicit model uncertainty escalates rather than guesses; policy-based — certain categories like security concerns, large refunds, or cancellation requests always escalate to a human regardless of how confident the model is, because those are business-risk decisions, not ML decisions; and sentiment-based — repeated frustration or an explicit request for a human escalates immediately with no added friction. When escalation happens, the human agent gets the full conversation, retrieved documents, and any account data already gathered, so the customer never has to repeat themselves.

Architecturally, an intent and risk classifier — deliberately a separate, lightweight model from the main conversational LLM — flags high-risk intents for fast-routing before the orchestrator even engages RAG or tools. The orchestrator manages multi-turn state and decides whether to retrieve help-center content, call read-only account-data tools strictly scoped to the requesting org_id, or escalate. Generation is grounded in retrieved passages and tool outputs, with the

agent explicitly disclosing it's AI and never inventing policy decisions like refund approvals it isn't authorized to make.

Evaluation covers groundedness and helpfulness via LLM-judge calibrated against periodic human review, plus escalation precision and recall specifically — did it escalate when it should have and avoid escalating when it shouldn't. The part I'd emphasize as a Staff-level insight: thumbs-down feedback and escalations get mined weekly into a prioritized content backlog for the human support-content team, because the agent's failures are often gaps in the underlying documentation, not the model. That closes the loop between the AI system and a human organization, which is what actually improves quality over time, not periodic model fine-tuning alone.

Key Takeaways — What the Interviewer Is Listening For

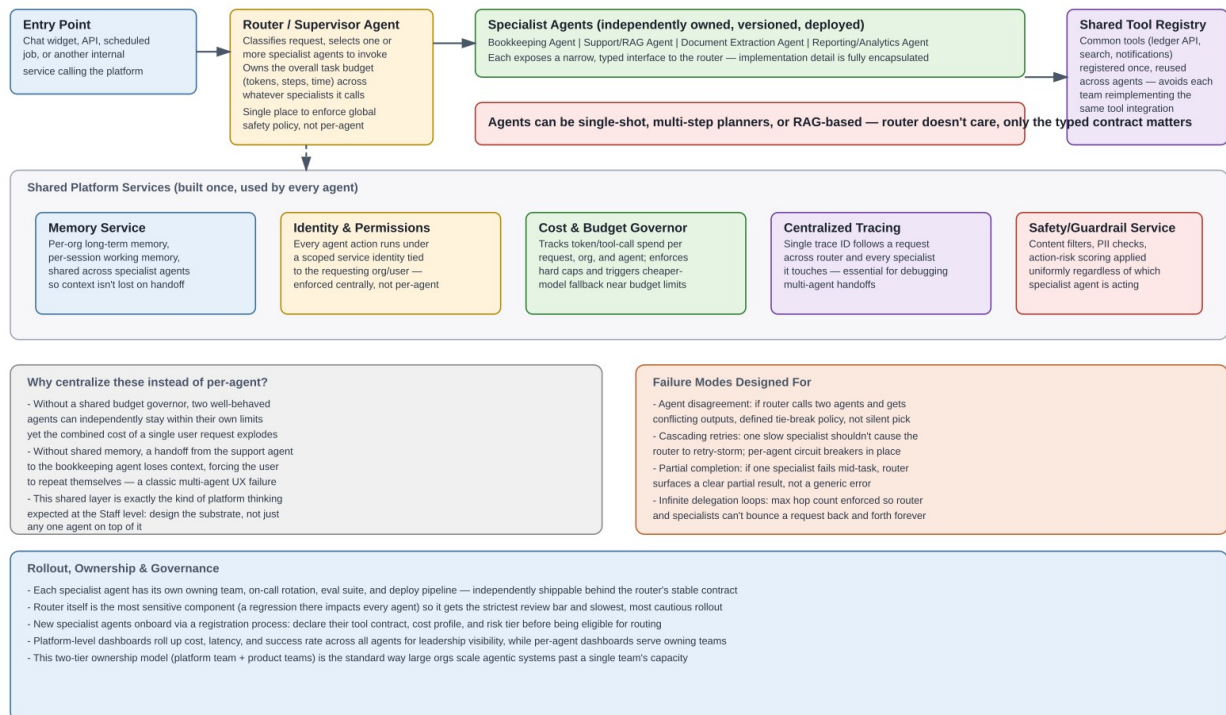
- Escalation policy — confidence-based, policy-based, and sentiment-based triggers — is the most important design decision in a support agent, design it first
- Use a separate, lightweight intent/risk classifier ahead of the main LLM to fast-route high-risk conversations
- Full context handoff to human agents (conversation + retrieved docs + tool outputs) avoids forcing customers to repeat themselves
- Evaluate escalation precision/recall explicitly, not just answer quality — wrongly avoided escalation is the costliest failure mode
- Agent failures (thumbs-down, escalations) should feed a content team's backlog — production quality here is a cross-functional feedback loop, not a model-only problem

Agent Question 5: Design a multi-agent orchestration platform with a router, specialist agents, shared memory, and safety controls

Tests: the highest level of platform thinking — designing the substrate multiple agent teams build on, centralized vs per-agent safety/cost controls, and failure-mode design for multi-agent systems. This is the most Staff-calibrated AI agent question in this set.

Solution Architecture

Multi-Agent Orchestration Platform — Router, Specialist Agents, Shared Memory, Safety



Full Answer

This question is about designing the substrate other teams build agents on top of, not designing any single agent — that framing is what separates a Staff answer from a senior one. A router/supervisor agent sits at the entry point, classifies the incoming request, and selects one or more specialist agents — a bookkeeping agent, a support/RAG agent, a document-extraction agent — each independently owned, versioned, and deployed behind a narrow, typed interface to the router. The router doesn't care whether a specialist is single-shot, a multi-step planner, or RAG-based internally; only the contract matters, which is what lets specialist teams ship independently.

Underneath, I'd centralize a small set of platform services rather than let every agent reimplement them: a memory service providing per-org long-term memory and per-session working memory shared across specialists so context survives a handoff; identity and permissions enforced centrally so every agent action runs under a scoped service identity tied to the actual requesting org; a cost and budget governor tracking spend per request across whatever specialists get invoked, because two individually well-behaved agents can still blow a

combined budget if nothing tracks the aggregate; centralized tracing with a single trace ID following a request across every agent it touches; and a safety/guardrail service applying content and action-risk checks uniformly regardless of which specialist is acting.

I'd explicitly design for multi-agent failure modes that don't exist in single-agent systems: a tie-break policy when two agents disagree, circuit breakers so one slow specialist doesn't trigger a retry storm from the router, graceful partial-completion handling, and a max-hop count to prevent infinite delegation loops between router and specialists. Governance follows a two-tier ownership model — the platform team owns the router and shared services with the strictest review bar since a regression there impacts everyone, while product teams own and independently ship their specialist agents behind a stable contract, onboarding through a registration process that declares each agent's tool contract, cost profile, and risk tier.

Key Takeaways — What the Interviewer Is Listening For

- The deliverable is the substrate (router + shared services) that lets many teams ship independent agents safely, not any single agent
- Centralize cost/budget tracking, memory, identity, tracing, and safety checks — per-agent enforcement of these guarantees nothing in aggregate
- Design explicitly for multi-agent failure modes: disagreement tie-breaks, cascading retries, partial completion, infinite delegation loops
- Two-tier ownership (platform team owns router/shared services; product teams own specialists behind a stable contract) is how this scales past one team
- The router is the most sensitive component in the whole system and deserves the strictest review and slowest rollout of anything in the platform

Whiteboard Session: Delivery Tips

Structure Your Answer Out Loud

- Restate the problem and ask 1-2 clarifying questions (scale, latency requirements, who consumes the output) before drawing anything
- Draw left-to-right following the data flow: ingestion -> features/retrieval -> model/agent -> action -> feedback loop
- Narrate the research/production boundary explicitly as you draw it — interviewers are listening for this specifically
- Always close with monitoring, failure modes, and one deliberate trade-off you made — this signals seniority more than a perfect diagram does

Common Follow-Up Probes to Expect

- "How would this change at 10x the current scale?" — have a scaling answer ready for your bottleneck component
- "What breaks first, and how do you know?" — tests whether you actually thought about failure modes or just drew boxes
- "Who owns this in production, and what's the on-call story?" — tests organizational/ownership thinking, the actual Staff-level differentiator
- "How would you A/B test or validate this before full rollout?" — tests production-readiness rigor beyond just shipping a model
- "What's the cheapest version of this that would still work?" — tests whether you default to the simplest viable design or over-engineer

Final Reminder

The diagrams in this guide are deliberately detailed so you can study the full component set, but on an actual whiteboard you will not have time to draw all of it. Practice a compressed version of each diagram — the 6-8 boxes that matter most — and use the detail in this guide to answer follow-up questions with depth once the core shape is on the board.